

SemaGraph

A deterministic, cache-resident kernel for classifying and compressing state-transition trajectories over a six-bit alphabet

Technical report — version 0.7 (`semagraph-64`)

June 2026

Abstract

SemaGraph is a deterministic kernel that represents observed change as finite trajectories over a small fixed alphabet and classifies and compresses those trajectories using only native integer operations. Each state is a six-bit word (Q_6 , 64 states); each direct transition is an XOR mutation mask; the complete transition basis is the $64 \times 64 = 4096$ ordered pairs. Per-transition metadata (Hamming distance, per-module distances, module scope, regime class) is a pure function of the mutation mask and is read from compile-time lookup tables with no data-dependent branch in the hot loop; the whole working set (≈ 4.4 KiB) is L1-resident. A struct-of-arrays batch interface processes ensembles of trajectories in one call and is written to auto-vectorize, with an optional explicit-SIMD path that reproduces the scalar output bit-for-bit. Trajectories over the eight-symbol alphabet Q_3 form a four-level hierarchy whose run-collapsed *shape* (with an aligned *dwell* vector) is the canonical form, while coarser projections (net mutation, endpoint code) serve only as pre-filters and never as form identities. The kernel performs no inference: states are produced by an explicit, versioned anchoring map from observations, and a language model may only synthesize policy or explanation over signatures the kernel has already computed. We give the formal model with short proofs, the computational architecture and its memory rationale, an exhaustive cross-implementation parity methodology, the evaluation hypotheses, the limits, and explicit non-claims (in particular: no complexity-theory claim, and no model of quantum evolution).

Sommario esteso

SemaGraph è un kernel deterministico che rappresenta il cambiamento osservato come traiettorie finite su un alfabeto piccolo e fisso, e che classifica e comprime tali traiettorie usando soltanto operazioni intere native. Ogni stato è una parola di sei bit (Q_6 , 64 stati); ogni transizione diretta tra due stati è una maschera XOR $m = s \oplus t$; la base completa delle transizioni è costituita dalle $64 \times 64 = 4096$ coppie ordinate. I metadati di una transizione (distanza di Hamming, distanze per-modulo, ambito di modulo, classe di regime) sono funzione esclusiva della maschera e vengono letti da tabelle di lookup costruite a tempo di compilazione, senza alcun salto condizionato sui dati nel ciclo caldo; l'insieme di lavoro (≈ 4.4 KiB) risiede in cache L1.

La tesi è volutamente ristretta: la parte *strutturale* del ragionamento sulle traiettorie è riconducibile a primitive elementari della CPU (XOR, popcount, lookup) e parallelizzabile su grandi insiemi (*ensemble*) di traiettorie, lasciando al modello linguistico soltanto il lavoro *semantico* (spiegazione, sintesi di policy). Il modello linguistico non calcola mai stati, maschere o firme. Le traiettorie sull'alfabeto a otto simboli Q_3 formano una gerarchia a quattro livelli la cui *forma* (lo *shape* ottenuto collassando le ripetizioni adiacenti, con il vettore *dwell* allineato) è l'identità di forma; le proiezioni più grossolane (mutazione netta, codice di endpoint) sono solo pre-filtri e non possono mai fungere da chiave di forma. Lo stato non viene inferito linguisticamente: è prodotto da una mappa di ancoraggio esplicita e versionata a partire dalle osservazioni.

Il documento fornisce il modello formale con brevi dimostrazioni, l'architettura computazionale e la sua motivazione rispetto alla gerarchia di memoria, la metodologia di verifica di parità esa-

tiva fra l'implementazione di riferimento in TypeScript e il core in Rust, le ipotesi di validazione, i limiti e le non-affermazioni esplicite. In particolare SemaGraph **non** è un motore fisico, non è un linguaggio di programmazione generale, non sostituisce solutori numerici o statistici, non avanza alcuna tesi di teoria della complessità (nessuna affermazione su P contro NP) e **non** modella l'evoluzione quantistica: quando applicato a hardware quantistico, opera unicamente sulle sequenze classiche di esiti di misura, cioè dopo il collasso.

Contents

Sommario esteso (Italiano)	1
1 Introduction and motivation	2
2 Formal model	2
2.1 The composite state space Q_6	2
2.2 Transitions, distances, scope	3
2.3 Regime taxonomy	3
2.4 Index and packed word	3
2.5 Chains and their compression	3
2.6 The four-level trajectory hierarchy over Q_3	4
2.7 Invariants and the comparison cascade	4
3 Deterministic anchoring	5
4 Computational architecture	5
4.1 Branchless, cache-resident classification	5
4.2 Struct-of-arrays batch and SIMD	5
4.3 Determinism, substrate, and interface	5
5 Verification methodology	5
6 Performance: architectural expectations	6
7 Relationship to known formalisms	6
8 Domains and evaluation hypotheses	6
9 Limits and threats to validity	7
10 What this is <i>not</i>	7
11 Future directions (speculative)	7
12 Conclusion	7

1 Introduction and motivation

Many complex systems produce *sequences* of states rather than isolated states. Organizational processes, financial models, discretized physical events and stochastic simulations can be observed as trajectories: an initial configuration changes, passes through intermediate regimes, and reaches a final configuration. The question is not only how to classify the current state, but how to represent the way the system got there.

The concrete problem that motivated SemaGraph was sampling cost. A stochastic simulation kernel required, for statistical robustness, more rollouts than consumer hardware could produce in reasonable time. The usual response — run more samples — attacks the statistics but not the use one makes of the samples already available. SemaGraph proposes a change of representation instead: collapse each trajectory to a canonical form and measure the distribution over *forms*, which stabilizes with far fewer rollouts than the distribution over raw paths, because the form space is coarser while still preserving the order of regimes visited.

The state alphabet is derived from the I Ching hexagram system, used solely because it is the oldest known combinatorial system that is natively binary: six positions, two values each, with a modular 3 + 3 structure. The cosmology is discarded; only the combinatorial skeleton is kept. What remains is Q_6 : 64 states, 4096 transitions, a complete basis for a six-dimensional Boolean space.

The thesis is deliberately narrow and is stated as such throughout: the *structural* part of trajectory reasoning reduces to native CPU primitives and parallelizes over ensembles, leaving only *semantic* work to any downstream language model.

2 Formal model

Notation: $B = \{0, 1\}$, $\oplus$ is bitwise XOR, $\text{popcount}(x)$ is the Hamming weight of x , and $[a, b)$ is a half-open integer interval.

2.1 The composite state space Q_6

$$Q_6 = B^6, \quad |Q_6| = 2^6 = 64. \quad (1)$$

A state $s \in Q_6$ is a six-bit word, stored as the low six bits of a byte; the canonical string form is `S64-` followed by six binary digits. Each state splits into two three-bit modules, written MSB-first as $b_5b_4b_3b_2b_1b_0$:

$$L(s) = (s \gg 3) \& 7, \quad U(s) = s \& 7. \quad (2)$$

The split carries no inherited meaning; it is the combinatorial property that lets a transition be classified as internal to one module or crossing both.

2.2 Transitions, distances, scope

For $s, t \in Q_6$ the *mutation mask* and *Hamming distance* are

$$m(s, t) = s \oplus t, \quad d(s, t) = \text{popcount}(s \oplus t) \in \{0, \dots, 6\}, \quad (3)$$

with per-module distances $d_L = \text{popcount}(L(s) \oplus L(t))$ and $d_U = \text{popcount}(U(s) \oplus U(t))$, and $d = d_L + d_U$. The *module scope* is `none`, `lower_only`, `upper_only` or `both_modules` according to which of d_L, d_U are positive.

2.3 Regime taxonomy

Let $\text{both} = (d_L > 0 \wedge d_U > 0)$. The regime class is the first matching clause, in this order:

condition	class (code)
$d = 0$	<code>no_change</code> (0)
$d = 1$	<code>bit_adjustment</code> (1)
$d \leq 2 \wedge \neg \text{both}$	<code>module_reconfiguration</code> (2)
$d = 6$	<code>full_bit_reversal</code> (5)
$d \geq 4$	<code>near_total_inversion</code> (4)
otherwise	<code>cross_module_regime_shift</code> (3)

Thus $d = 2$ confined to one module is `module_reconfiguration`, while $d = 2$ split across both modules, and every $d = 3$, is `cross_module_regime_shift`. The taxonomy is a total function of (d, d_L, d_U) ; the integer codes are part of the cross-implementation contract.

2.4 Index and packed word

The $64 \times 64 = 4096$ ordered pairs form the complete direct basis (the 64 identities included; $64 \times 63 = 4032$ are non-identity). The canonical index is

$$I(s, t) = 64s + t \in [0, 4096). \quad (4)$$

For low-overhead FFI the full record is packed into one 64-bit word, least significant field first:

source:6		target:6		mutation:6		distance:3		d_L:2		d_U:2		scope:2		regime:3		index:12
----------	--	----------	--	------------	--	------------	--	-------	--	-------	--	---------	--	----------	--	----------

This word is verified bit-for-bit against the reference for all 4096 transitions.

2.5 Chains and their compression

A chain is an ordered sequence $C = (s_0, \dots, s_n)$, each step yielding $m_i = s_{i-1} \oplus s_i$ and the ordered signature $(m_1, \dots, m_n)$.

Proposition 1 (Net mutation telescopes). $M_{\text{net}} := m_1 \oplus \dots \oplus m_n = s_0 \oplus s_n$.

Proof. $\bigoplus_{i=1}^n (s_{i-1} \oplus s_i) = s_0 \oplus s_n$, since every interior s_i occurs exactly twice and cancels under XOR. $\square$

Hence M_{net} depends only on the endpoints and discards order: distinct chains can share a net mutation, which is precisely why the ordered signature and the net mutation are retained as distinct objects. The *cumulative distance* $D_C = \sum_i \text{popcount}(m_i)$ measures total traversal effort (it does not cancel out-and-back moves). The operative signature is $\Sigma(C) = (s_0, s_n, (m_1, \dots, m_n), M_{\text{net}}, D_C)$; it indexes the raw chain but never replaces it.

2.6 The four-level trajectory hierarchy over Q_3

Trajectories are typed over the eight-symbol alphabet $Q_3 = \{0, \dots, 7\}$.

Level 0 — alphabet. The eight elementary states.

Level 1 — oriented edge T_{64} . The ordered pair $e(a, b) = 8a + b \in [0, 64)$.

Proposition 2 (Recoverability). e is injective and orientation-preserving: $a = \lfloor e/8 \rfloor$, $b = e \bmod 8$, and $e(a, b) = e(b, a)$ iff $a = b$.

T_{64} shares the cardinality 64 with a Q_6 point but is an oriented $Q_3 \times Q_3$ edge; the two are distinct types and never interconvert.

Level 2 — paths, run-collapse, dwell. A path is a word over Q_3 . Its run-collapsed *shape* merges adjacent duplicate states; non-adjacent repeats are preserved (so $[0, 1, 0, 3]$ does not collapse and stays distinct from $[0, 1, 3]$). The *dwell* vector records, in run order, how many positions each run held.

Proposition 3 (Lossless decomposition). $\text{path} \mapsto (\text{shape}, \text{dwell})$ is a bijection onto pairs in which *shape* has no two adjacent entries equal and *dwell* is a positive-integer vector of equal length.

Proof. This is run-length encoding restricted to maximal constant runs. The no-adjacent-equal condition makes run boundaries unique, so repeating $\text{shape}[k]$ exactly $\text{dwell}[k]$ times inverts the map. $\square$

Level 3 — declared lossy projections. Each with its collapse factor:

projection	image size	role
net mutation (Q_3)	8	coarse pre-filter only
endpoint code (T_{64})	64	medium pre-filter only
endpoint Hamming	scalar	magnitude only
shape_key	injective on shape	form identity
exact_key	injective on path	trajectory identity
dwellexsignature	—	duration identity

2.7 Invariants and the comparison cascade

Proposition 4 (Endpoint soundness). *Equal run-collapsed shape implies equal endpoint.*

Proof. Run-collapse never removes the extreme positions, so the shape’s first and last entries are the path’s first and last states, and the endpoint is a function of those two. $\square$

The converse fails: $[0, 1, 3]$ and $[0, 2, 3]$ share the endpoint $e(0, 3)$ but differ in shape. Hence the endpoint is a *sound pre-filter* (a mismatch proves the shapes differ) but **never** a form decision. The binding invariant is that *no* lossy projection is a form key: **shape_key** decides form. The comparison of two trajectories returns exactly one relation:

result	condition
different_trajectory	endpoints differ (short-circuit)
different_pattern	else shapes differ
equivalent	else dwell signatures equal
same_form_different_duration	otherwise

3 Deterministic anchoring

States are not inferred by a model. They are produced by an explicit anchoring map under a versioned rule set $R = (r_1, \dots, r_6)$:

$$A_R : X \rightarrow Q_6, \quad b_i = r_i(x) \in \{0, 1\}, \quad (5)$$

where X is the space of observed/parametric inputs and each r_i is a declared threshold, Boolean condition or predicate. Given the same x and the same R , $A_R(x)$ is identical (replayability). Measurement uncertainty is recorded at the measurement layer — tolerance, unit, source, timestamp, rule version — and does not enter the transition kernel. A language model may explain a signature, propose policy, or compare approved policies; it must never fabricate observations or alter states, masks or signatures. This is the anti-inference boundary.

4 Computational architecture

4.1 Branchless, cache-resident classification

Every scalar the classifier needs is a function of the six-bit mask $s \oplus t$, so the decision surface fits in a few compile-time tables indexed by that mask. No **if/match** on the data appears in the hot loop. The working set is

table	entries	bytes
distance / lower / upper / scope / regime (by mask)	5×64	320
regime by transition [<i>u8</i> ; 4096]	4096	4096
total		4416

At ≈ 4.4 KiB the tables occupy roughly 14% of a typical 32 KiB L1 data cache, so once warm the common path is a few L1-latency loads plus an XOR and a multiply-add, with no branch to mispredict. The tables are built from the same `const fn` classifiers as the scalar path, so they cannot drift; an exhaustive test asserts equality on all 4096 transitions.

4.2 Struct-of-arrays batch and SIMD

The batch interface takes parallel input arrays (sources, targets) and writes into caller-provided output columns, as a flat counted loop with no early return, no per-iteration allocation and no data branch. This is the layout an auto-vectorizer can widen. An optional, non-default `simd` feature provides an explicit SSE2 path that produces output identical to the scalar batch (asserted by test); the scalar version is the source of truth. The trajectory batch compresses an ensemble in one call into preallocated columns.

4.3 Determinism, substrate, and interface

The classification path uses no floating point and no randomness. The kernel is implemented in dependency-free Rust with a C ABI (and is WASM-compatible); the TypeScript implementation remains the executable specification. A single zero-dependency CLI, `semagraph`, exposes the operations; its `-json` output is a stable contract designed for consumption by automated agents (data on stdout, errors on stderr, conventional exit codes; NDJSON streaming for batches).

5 Verification methodology

Parity is exhaustive, not sampled. The TypeScript reference generates a golden fixture describing all 4096 transitions with canonical integer codes and the packed FFI word; a dependency-free Rust test asserts the core reproduces every entry exactly. A second, seeded fixture covers Q3 trajectory projections (shape, dwell, net mutation, endpoint, endpoint Hamming, cumulative distance), and the branchless path is tested against the validated path on every transition. Continuous integration regenerates both fixtures from the reference and fails on any drift, so the two implementations cannot diverge silently.

6 Performance: architectural expectations

We state expectations, not measurements. A Criterion benchmark harness compares the naive per-item path, the branchless table path, the scalar batch, the optional SIMD batch, and end-to-end trajectory compression over a seeded ensemble, reporting transitions/second and trajectories/second. The branchless path is expected to be at least as fast as the naive path (constant-time L1 loads replacing a comparison chain); the batch is expected to meet or exceed the per-item path (vectorizable shape); the SIMD gain is bounded by the scalar table gather and is a demonstration, not a claim. All four paths produce identical output, which is tested rather than assumed. Concrete numbers belong in the repository’s `BENCHMARKS.md`, recorded with CPU, compiler flags and seed; they are deliberately not invented here.

7 Relationship to known formalisms

Family	Similarity	Difference
Finite automata	Finite states and transitions	Oriented toward compressing <i>observed</i> trajectories, not recognizing languages
Statecharts	Guarded transitions	Does not orchestrate application/UI behaviour
Timed automata	Sequences with durations	Dwell is a discrete count, not a real-valued clock; no timing constraints
Rule engines	Predicates and rules	Produces no general inference; it classifies and compresses
Policy-as-code	Formalizable operational rules	Policy is strictly downstream of the computed signature
Markov-switching / Monte Carlo	Trajectories and regimes	Does not sample paths; it deterministically classifies given paths

8 Domains and evaluation hypotheses

The framing is domain-neutral: a deterministic compression layer for state-transition trajectories where sampling cost is the bottleneck and path-dependence is structural. Worked translations accompany the repository for (i) stochastic simulation (grouping rollouts by `shape_key` to stabilize a histogram over forms); (ii) regime-switching Monte Carlo and path-dependent instruments, where in non-ergodic settings the ensemble mean does not equal the time average and the distribution over typical trajectories is what matters [7, 8]; (iii) classical analysis of measurement-shot sequences from quantum hardware (see the non-claim in §10); (iv) decision-making under deep uncertainty, where the anchoring map is the quantization of qualitative parameters. The testable hypotheses are: shape/signature processing reduces computation versus full-trajectory processing; signature-keyed policy reuse improves consistency; the information lost to compression is bounded and its false-equivalence cases are characterizable; and the set of domains admitting direct deterministic anchoring (versus those requiring an uncertainty envelope first) can be delineated.

9 Limits and threats to validity

The system’s quality depends on the anchoring rules: a poor A_R yields meaningless states regardless of the kernel. Compression is lossy, and the most critical risk is *false equivalence* — distinct chains sharing a net mutation or even an endpoint. This is mitigated structurally (the form decision uses `shape_key`, not a lossy projection) and operationally (the raw chain is always retained and the signature only indexes it), but it cannot be eliminated. The kernel does not replace numerical or scientific solvers.

10 What this is *not*

SemaGraph is not a physics engine, a general-purpose programming language, a universal simulator, or a replacement for statistical/numerical models. It advances *no* complexity-theory result and makes no claim about P versus NP; it is an engineering reduction of a narrow, structural

task to native CPU primitives. It does not model how the state axes are learned — they are stipulated by the anchoring map, not emergent. When applied to quantum hardware it does **not** model quantum evolution (continuous unitary dynamics, complex amplitudes): it operates only on the classical sequences of measurement outcomes after collapse, as a classical signal-analysis layer.

11 Future directions (speculative)

Wider portable-SIMD and multi-lane implementations of the batch path; an FPGA/lookup-ROM realization of the classifier as a near single-cycle lookup over the 12-bit transition index; and learned anchoring — inferring the state axes from data rather than stipulating them — which is the genuinely hard, deliberately out-of-scope problem.

12 Conclusion

SemaGraph formalizes a deterministic kernel that compresses observed change into reproducible structural signatures over a finite 64-state Boolean space. Its contribution is architectural honesty about a narrow claim: the structural part of trajectory reasoning is branchless, cache-resident and vectorizable, and can be separated cleanly from the semantic work left to a language model. The mathematics, the memory rationale, and an exhaustive parity methodology are given so that mathematicians, engineers and physicists can each evaluate the part that concerns them.

References

- [1] C. E. Shannon, “A Mathematical Theory of Communication”, *Bell System Technical Journal*, 1948.
- [2] R. W. Hamming, “Error Detecting and Error Correcting Codes”, *Bell System Technical Journal*, 1950.
- [3] J. E. Hopcroft, R. Motwani, J. D. Ullman, *Introduction to Automata Theory, Languages, and Computation*, Pearson, 2006.
- [4] D. Harel, “Statecharts: A Visual Formalism for Complex Systems”, *Science of Computer Programming*, 1987.
- [5] R. Alur, D. L. Dill, “A Theory of Timed Automata”, *Theoretical Computer Science*, 1994.
- [6] U. Drepper, “What Every Programmer Should Know About Memory”, technical report, 2007.
- [7] O. Peters, “The ergodicity problem in economics”, *Nature Physics*, 2019.
- [8] J. D. Hamilton, “A New Approach to the Economic Analysis of Nonstationary Time Series and the Business Cycle”, *Econometrica*, 1989.
- [9] S. Russell, P. Norvig, *Artificial Intelligence: A Modern Approach*, Pearson, 2021.